

Introduction →

polished. write as I go. My thoughts on the paper. Some insights may be contained on separate scraps of paper!

- This journal is a log of my journey programming an analysis tool for genomic sequences. I always wanted to invent something and 2 days ago I stumbled across a very interesting "research paper" on Github. It was 3 days old at the time and wasn't very popular. I wouldn't do myself justice if I haven't started this project after reading the contents of the paper.
- The idea? Modelling language as a Continuous Physical field ruled by wave equations. Instead of the "textbook" methods of LLMs I've been taught (discrete sequence of tokens), Wave Field V3.5 scatters token info onto a continuous field. Propagation using physics based wave dynamics and then gathering it to make predictions. This is genuinely new architecture and could lead to major breakthroughs.

From Paper

Wave kernel: Each attention head is parameterized as a damped oscillation

$$k(t) = \exp(-\alpha \cdot t) \cdot \cos(\omega \cdot t + \psi) \text{ for } t \geq 0 \text{ (causal)}$$

ω (frequency) → oscillation speed → Attention pattern periodicity

α (damping) → Decay rate → How far back to attend

ψ (phase) → offset → Head diversity

→ Fast Fourier Transform

Convolution computed through FFT in $O(n \log n)$

- Falls within 5% of standard transformer with WikiText-2, character tokenizer, 30 epochs, same hyperparameters. **5.9 vs 6.2**
- But what is it that excites me about this tech? What are the limitations? Explored on next pg...

24/2/26

The PositivesTesting Performed in
Research Paper

Nr1

COMPUTATIONAL SAVINGS AT SCALE

<u>Sequence Length</u>	<u>standard $O(n^2)$</u>	<u>Wave ($O(n \log n)$)</u>	<u>Savings</u>
512	134M ops	14.3M ops	9x
2048	2.1B ops	68M ops	31x
8192	34B ops	319M ops	107x
32,768	550B ops	1.5B ops	367x

Nr2

A NEW APPROACH

(operations etc)

Tokens live on Continuous field and info propagates via damped wave equations so each head has the 3 parameters to learn, as I said on last pg (frequency, damping, phase). Beautiful thing is that these heads/waves self-organise into roles. stuff like local grammar, medium range context, long-range document structure. But best of all, it's physics based. Diagnostics are so much easier than your run of the mill LLM (if you are knowledgeable you may know what kind of black box that can be). The author of the Github post describes finding bugs by inspecting physical quantities (energy flow, conservation, causality), not by guessing → Wow! this could do so much for innovating AI and approach AGI

Q

Example

Truly amazing.

V.3.1 → Conservation shortcut bug → Solved through: Energy flow trace showed residual amplification

In the paper the author lists these Bug Fixes and the physics used from V3 to V3.5 on a table. Very Awesome Sauce.

Limitations

And why I am approaching this project despite them

The BPE Bottleneck (Byte-Pair Encoding):

from their research came a problem that is very critical. When scaled to a more realistic BPE of ~8000 tokens, the performance fell off drastically! Cratered. PPL (Perplexity) went from near parity (6.2 vs 5.9) to being 87% worse! (170.7 vs 91.4) Very Poor against the small scale test @ ~200 tokens.

Why does it happen?

This is an info bottleneck at its core ~~problem~~. As we know a transformer performs its calculations by letting each token to talk (from my course; it takes input text, converts it into a vector of embedding that has meaning $\begin{bmatrix} 0.1 \\ 0.6 \\ 0.3 \\ 0.7 \end{bmatrix}$ in high dimension space. Through matrix multiplication it calculates the dot products that measures the alignment of tokens (input) and so on with Key, Query, Value Subspaces, or Feed Forward). The Matrix multiplication allows for each token to interact with one another, but Wave Field works off a completely diff principle. Info is scattered onto a field, processed, then gathered. For small vocab this process that gets rid of unnecessary info is fine but for a large Vocabulary requiring fine attention to detail distinctions, the field can't carry enough info for the ability to make distinctions with accuracy. That's a mouthful. Anyways, the test model is simply too small for an 8k vocab. 6 million parameters with 256-dimensional embedding is not enough. Compression ratio of 31:1 is too high. Bandwidth is required (higher dimensional embeddings) to represent that many concepts. Yes the team is now training a new model @ 100M parameters and this'll be a make or break moment for this tech we in the AI space, but this will take time and compute, and as we know, this is a scaling issue. If their performance gap with BPE narrows significantly, it will be such a validation of Wave approach. If the gap remains, maybe there's just a fundamental limitation of current approach with wave field.

24/2/26

What's up with DNA:

I did research after reading this paper ^{into concepts with low vocab} and came across DNA. It only has a vocab of 5 nucleotides (A, C, G, T, N). This is even smaller than the character level tokenizer where field achieved close Performance with Transformers (~200). This aligns with my values and I feel very passionate about using math to solve real world problems. I want to combine Data Science and Biomedical engineering to save peoples lives, maybe help develop devices that prevent or detect diseases in early stages. This passion translates itself onto this project of mine. If I do not succeed then at least it's something for the portfolio when applying to positions that help me attain this goal. To have an impact, a positive impact on people. In DNA vocab, the bottleneck of Wave Field tech disappears. DNA stretches for ~~long~~ long strings so a model that saves on compute compared to transformers will be revolutionary, I hope. With this attention retention I'll be able to detect genes controlled by promoters thousands of bases away. Other ~~potential~~ potential possibilities are:

- Enhancers Can be hundreds of thousands of bases away → Even on diff chromosomes
- Chromatin folding 3D structure brings distant sequences together
- Non-coding RNA's long molecules with complex secondary structure
- ⇒ Wave-like properties of DNA actually might be picked up by the wave field process. DNA physical structure has this pattern. So wave dynamics might be capturing actual biological physics

Key thing is the distance / Length. Wave field's $O(n \log n)$ complexity would handle these ~~long~~ long sequences way more efficiently than a transformers $O(n^2)$. ((squared) / (exponent))

Periodic Patterns are Fundamental to DNA. In ~~the~~ protein-coding regions the codon structure has a 3 base period. Similarly, Nucleosome positioning has an about 10.5 base period. Basically There's a chance that the wave kernels with learnable frequencies could automatically discover these structures with enough data.

So what am I aiming for?

Possibilities are endless so I narrowed it down to 4 projects ranging in ascending difficulty and time. I'll approach it ~~as~~ as a level-up system. Level 1 is easy and level 4 is endgame, similarly to video games. If I can't pass a level I cannot move onto the next. I will simply leave it alone and focus on other life goals. Maybe coming back with a changed perspective on the problem. No pressure. I do not plan on making any money from this. This is experience, fun and life goals. Criss Cross Apple sauce.

Level 1: Sequence Classification (Predictor) Start off

Goal: Predict whether a DNA Sequence has a certain property.

Examples:

- Promoter Prediction → Given a DNA Sequence, does it contain a promoter? Where?
- Splice Site prediction → Identify exon/intron boundaries
- Enhancer prediction → Identify regulatory elements
- CpG island prediction → Find regions important for gene regulation

How will I measure my success → Match or EXCEED CNN-based approaches (the current standard for DNA) like Basset or DeepSEA.

Level 2: Sequence GENERATION Ambitious Realistically I stop here

Goal: Generate realistic DNA sequences with desired properties

DNA has strong higher-order structure. How will this be caught by wave Field?

Examples:

- Design synthetic promoters with specific strength
- Generate regulatory sequences that drive expression in specific tissues
- Optimize protein-coding sequences for expression, while maintaining amino acid sequences

24/2/26

Level 2

Implementation:

- Use existing autoregressive generation setup (like the test generation)
- Train on chunks of genome sequence
- Condition on desired properties (tissue, expression level etc...)

Level 3: Functional Prediction (Cutting-Edge) Wann

Goal: Predict the functional effect of genetic variations

Examples

- expression quantitative trait loci → Single Nucleotide Polymorphism
- eQTL prediction: does SNP affect gene expression?
- Disease variant effect: is a mutation pathogenic
- Chromatin accessibility: does a sequence open chromatin

Implementation:

- Train model to predict functional measurements from sequence
- For variants: compute the diff in prediction between reference and alternate allele
- Compare to existing tools like Enformer (Transformer based tech)

↓
This could be where Wave Field shines. Enformer uses a Transformer with ~200kb context (197 from Research but not sure) which is computationally massive. Wave Field could maybe handle longer context AND less compute. Vision is there.

24/2/26

This is the stuff LEVEL 4: Multi-Species Comparative Genomics ^{level} PHD Staff

Goal: Understand evolutionary conservation and divergence

Examples:

Not sure about this

Idea

Must revisit

probably

requires a lot of Biology knowledge that I do not yet possess

- Align multiple species genomes using wave dynamics
- Identify conserved non-coding elements (CNEs)
- Model evolutionary rates across the genome

Approach:

- Treat each species as a diff "view" of same ancestral sequence maybe
- Use field to represent the ancestral state
- Model mutations as disturbances to movement/behaviour of the field

And with this exploration I have determined my goals and challenges. Time to get started. Locked

What is the diff between what I can and can't yet?

(SAAS) Stride AI

Wave Field DNA

- | | | |
|-------------------------------|---|--|
| • Need to support users | ↔ | • Prove my concept |
| • Uptime matters | ↔ | • Science papers accepted with failures = 5 |
| • Feature requests | ↔ | • Single focus |
| • Customer support | ↔ | • Just me and the data |
| • Competition | ↔ | • No one, blue ocean |
| • Known requirements | ↔ | • Research (unknown if it works) |
| • Clear user needs | ↔ | • No users yet (just science) |
| • Python + FastAPI + SQLite | ↔ | • Fourier Transforms + wave physics + genomics |
| • Debug with print | ↔ | • Need physics-based diagnostics |
| • Existing patterns to follow | ↔ | • <u>No one</u> has done this |

24/2/26

Understanding the Wave field

24/2/26

THIS IS NOT A Transformer. Wave field happens in 2D, not higher dimension dot products like transformers. Only similarity is the embedding process at the start. Vectors!

DNA letter A $[-0.8, 0.1, 0.2, -0.3, 0.4 \dots]$

DNA letter C $[0.5, -0.6, -0.7, 0.4 \dots]$

The network needs to represent them differently. The embeddings definitely won't need to have as much dimension as in transformers since wave equation itself will do the work.

Now to visualise this information... It's gonna be hard.

- Vectors are still needed to embed properties of each base. Needed for initialising physical fields.

Mental Model 1?

SO DNA is 1D ... A-T-G-C-A-T-G-C (just position along strand)

But waves need to transfer "energy", essentially $\psi(\omega, \psi, \alpha)$. We need this to have properties like interference, coupling (interacting waves), patterns. So this will be a 2D process. Physics based application of token attention.

Y-axis is activation at each point? will this be it? X-axis is the position across the DNA sequence.

So if I take An example strand ATC then it'll look like...

	0	1	2	3	→
	A	T	C	...	

24/26

OH SHIT... The y -axis could then have a similar system of associative values. Not a single measure. Counting upwards the neural network could learn to associate values with diff properties.

etc
chemical influence? 2
methylation status? 1
chemical properties? 0

So then when I "drop" a token, its embedding for $\vec{A}$ is $[0.2, 0.8, -0.3, 0.5, \dots]$

$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow$
 $y=0 \quad y=1 \quad y=2 \quad y=3$

(vertical position)

So then at $x=0$, (position of $\vec{A}$)

the code can set:

field $[0,0] = 0.2$

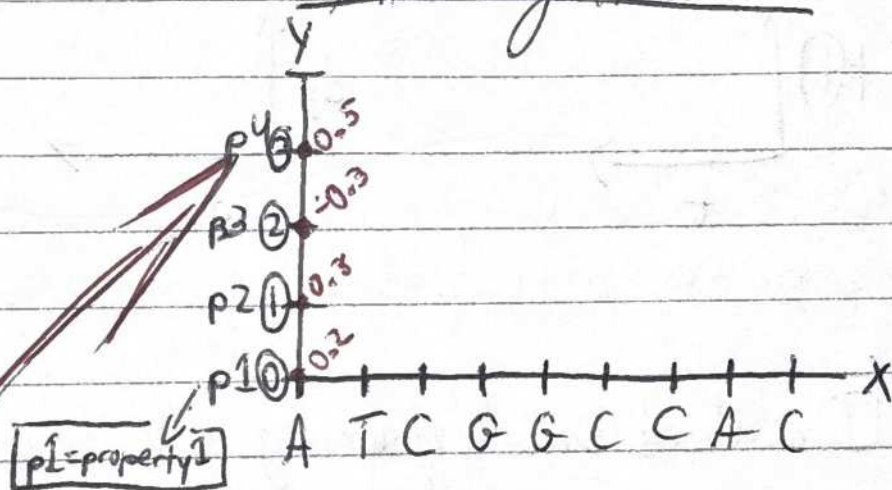
field $[0,1] = 0.8$

field $[0,2] = -0.3$

field $[0,3] = 0.5$

... and so on

All Together:



So then after all that the waves start to spread y -axis (vertically) and horizontally (x -axis)

This is actually perfect!
think about it. DNA

parameters 1000s of
"x" away will be affected
by an enhancer at pos 0
if trained correctly. Wave
carries signal down the strand.
The model works.

Yes. This is like the matrix dot product of transformers. Info travels to neighbouring letters across the x -axis and along the y -axis features will mix or interfere with each other.

Then upwards across the y -axis there'll be an interaction of the features/properties. Chemical properties influence methylation etc. Waves allow features to "talk".

24/2/26

But Wait!

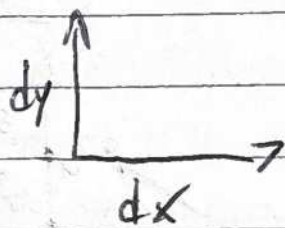
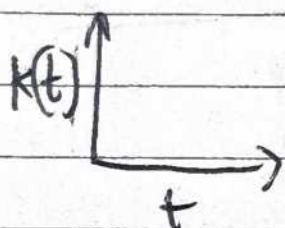
3/2/26

After looking at the original paper's ~~exp~~ $k(t) = \exp(-\alpha t) \cdot \cos(\omega t)$ this isn't ~~feasible~~ ~~feasible~~ compatible with my mental model what I need would be wave kernels in both x and y directions

lets modify the papers original equation. As much as I hate this approach, it makes so much sense for DNA.

distance y, distance x

so maybe dy, dx shorthand?



~~How will I approach this?~~

how will I approach this?
2 diff equations or
1?

I realised something wrong with the original formula. Not really but for my use case yes.

$$\exp(-\alpha \cdot t)$$

since t is a value from my vector embedding process it can range from $[-1 \leq t \leq 1]$. The negative values counter act the negative sign in front of alpha. This will ~~destroy~~ the damping property and create so much noise in my wave field that nothing will make sense.

Maybe Mental Model I had a faulty approach from my end.

$$\exp(-\alpha \cdot |t|)$$

so I will have to use modulus for absolute value. But will this skew the meaning attached to the vector?

Mental Model 1 is saved.

Analysis of
Function

I was too hasty to give up. When analyzing the rest of the function ~~is~~ ^{is} ~~positive~~

$\cos(-A) = \cos A$ from log tables but... ^{(meant to be $\phi(\psi)$)} ψ changes things

$e^{-\alpha t}$ this has an effect of multiplying by a lower and lower decimal

Good enough for me! Embedding preserves meaning in negative values

$\cos(-\alpha dx + \psi) \neq \cos(\alpha dx + \psi)$
except for $0!$

So this acc means that Amplitude (e^0 or $\exp()$) with $|dx| \rightarrow$ just distance
Phase ($\cos$) uses $dx \rightarrow$ preserve direction

How far info travels depends only on distance and what Direction it came from effects the Pattern!

So Basically I'm stupid because parameters hold multiple functions just like in Transformers ^{based networks}. What did I expect! At the end of the day their both networks.

So one ^{single} embedding really does:

- Initial amplitude at position (see eggs back)
- Part of wave source
- Affected by damping ($|dx|$ preserves magnitude)
- Affects phase relationships (signs in dx matters)
- Combines with other embeddings via interference.

All this is just one nr in the vector embedding.
Wave field is cool

I'm seeing a pattern to transformers...

25/2/26

Generalizing the Equation for Both Directions

$$k(d) = \exp(-\alpha \cdot d) \cdot \cos(\omega \cdot d + \varphi) \quad d \text{ is direction}$$

dx = distance x-direction (position)

dy = distance y-direction (property/feature)

Must interact so 2 waves needed

$$k(dx, dy) = \exp(-\alpha_x |dx| - \alpha_y |dy|) \cdot \cos(\omega_x dx + \varphi_x) \cdot \cos(\omega_y dy + \varphi_y)$$

subscript to differentiate term

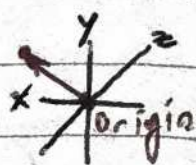
using exponent rules terms can be separated. This means Decay is independent between properties. Perfect!

But wait! If I separate exponents...

$$\begin{aligned} & \exp(-\alpha_x |dx|) \cdot \exp(-\alpha_y |dy|) \cdot \cos(\omega_x dx + \varphi_x) \cdot \cos(\omega_y dy + \varphi_y) \\ &= [\exp(-\alpha_x |dx|) \cos(\omega_x dx + \varphi_x)] [\exp(-\alpha_y |dy|) \cos(\omega_y dy + \varphi_y)] \end{aligned}$$

$k(dx, dy) = k(dx) \cdot k(dy)$ wow what a beautiful property, this is incredibly useful generalization that allows to keep the $O(n \log n)$ and Fast Fourier Transform strategy of research paper.

Vector embedding does not hold the same function for Wave Dynamics as it does for Generative pre-trained Transformers. Embedding in GPTs give a vector that has distinct direction due to the embeddings

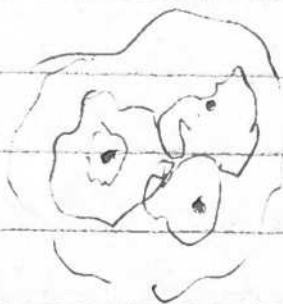


But for Wave Dynamics the embeddings behave more like a list $[]$. This is more similar to the computer science definition of a vector.

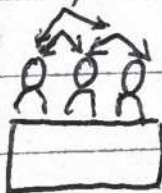
Visualising the Equation

A nice visualisation of Wave dynamics information transfer is an analogy of a pond, like a pond, and our model learns to drop pebbles (kernel heads) at specific points in the pond simultaneously.

The ripples in the pond interact, certain frequencies will amplify, and certain ones will diminish. The resulting field on the pond will be a result of the information transfer of the Kernel heads. This analogy is actually perfect since it uses a physics engine to transfer info through waves. Both in theory and analogy.



This process is so much more efficient than Transformers where each token has to talk individually. In transformers the info carry looks like a conference room where each token has to "talk" to each token ~~one~~ in order to transfer info. You can see how these analogies highlight the limitations and upsides of the methods. Imagine 8000 people (tokens) in a conference room, trying to exchange info individually to create a network. Now increase it by 8000 people to make it 16,000. This is an exponential increase in compute as each one has to talk w/ each one.



So how would training such a model look like? I think backpropagation would still work here but the ~~forward~~ flow/process would look different. Let's list the number one diff, that being the physics "engine". The physics of the wave interactions CANNOT be trained/changed. This means that the resulting waves ^{field} will be the output of the forward pass. Think of it as the pond, again. If we throw pebbles into the ~~like~~ pond at random positions

26/2/26

2/2/26

and then measure the waves afterwards, we can compare them to the resulting waves of training data and adjust for this by essentially throwing the pebbles in diff locations. This is perfect and further solidifies the link between the maths and physics of this token attention process. So we can control the positions of our pebbles but can't we also control the type of pebbles? ~~That would be the~~ Thinking about it now, the embedding vector process will be responsible for both local properties and how ~~the~~ the nucleotide effects other nucleotides because of its position AND properties. Wow. This is just the right architecture for DNA sequence analysis and potentially generation. With the right approach this could be used to code genomes with desired properties. The increased attention and decreased compute could possibly even help with analysing causes of Genetic Disease!

On github I wrote a simple programme that maps a single instance of the function $k(dx, dy)$ and the resulting graph results in a ~~2D~~ 3D surface with a ripple effect. A maximum in the centre (result of the cosine wave) signifying the heat and local importance of info to be transported. Going outwards the surface has a "ripple" effect, parallel to my pond analogy. I will print this sample graph and paste it into this journal.

Back to Backprop

The ~~same~~ differential equations and chain rule of backprop are parallel to the differential equations of fluid dynamics. Interesting to see the similarities. Maybe this will prove useful in the future.

Now let's investigate the backprop architecture...

I think the best way to approach this is by doing a worked example, bring a number on a journey, and derive a generalisation.

Let's suppose I have a sequence of A-T-G. I'll also assume 2 properties to keep it visual. 1 attention head and 1 wave propagation step. Really ground 0, tryna get the basics down.

My Variables are:

- Embedding matrix $\leftarrow$ Learned
- $\alpha, \omega, \varphi \leftarrow$ Learned
- output weights $\leftarrow$ learned
- Loss

Ok so step 1 would be a forward pass

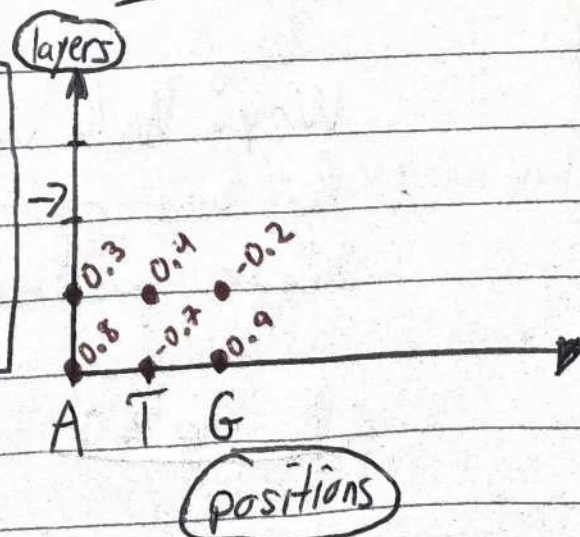
Input Look up embeddings

A $\rightarrow$ [0.8, 0.3]

T $\rightarrow$ [-0.7, 0.4]

G $\rightarrow$ [0.9, -0.2]

plot onto
layers of y
axis and
positions
of x-axis



Now, step 2 is the wave propagation through the equation. Not enough room, I'll explore this on next page $\rightarrow$

27/2/26

Step 2 Calculating the kernels values

I'll need to set the learned parameters to values for the minute. Let values be:

$\alpha_x = 0.5$ (damping along DNA)

$\alpha_y = 0.3$ (damping between property layers)

$\omega_x = 2$ (frequency along DNA)

$\omega_y = 1.5$ (frequency between property layers)

$\psi_x = 0$ (phase along DNA)

$\psi_y = 0.5$ (phase between property layers)

$$k(dx, dy) = e^{(-\alpha_x |dx| - \alpha_y |dy|)} \cdot \cos(\omega_x dx + \psi_x) \cdot \cos(\omega_y dy + \psi_y)$$

$$k(dx, dy) = e^{(-0.5 |dx| - 0.3 |dy|)} \cdot \cos(2dx + 0) \cdot \cos(1.5dy + 0.5)$$

~~definition~~

~~the kernel is a function that takes two inputs, dx and dy, and returns a value between -1 and 1. It is used to calculate the similarity between two positions in a sequence.~~

Now time to calculate for $dx = [-2, -1, 0, 1, 2]$ and $dy = [-1, 0, 1]$

Why? Well we need to look at positions coming from the left and right direction. More clearly, the waves coming from these positions.

Example:

- Position "n" itself $\boxed{dx=0}$
- Position "n-1" $\boxed{dx=-1}$ ← wave came from left
- Position "n+1" $\boxed{dx=+1}$ ← wave came from right
- Position "n-2" $\boxed{dx=-2}$ ← wave came from further left

I forgot to mention that the equation uses radians radians are clean when derived

$$d/dx \sin x = \cos x$$

$$d/dx \cos x = -\sin x$$

meanwhile degrees:

$$d/dx \sin(x^\circ) = (\pi/180) \cos(x^\circ)$$

since backprop is all about derivatives, radians keep the math clean!

2/2/26

∴ If I had the sequence

0 1 2 3
A T G C

← computing here

To compute wave at position 2 (G):

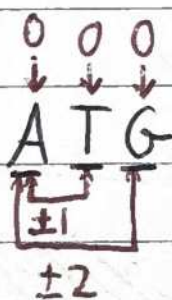
- Look left, position 1 contributes with $dx = -1$ (T)
- Look further to the left, position 0 contributes with $dx = -2$ (A)
- Look to the right, position 3 contributes with $dx = +1$ (C)

The kernel $k(dx)$ says: ~~how much~~

- $k(-2)$ = how much does a source/nucleotide 2 steps left affect here?
- $k(-1)$ = how much ... 1 step left affect here?
- $k(0)$ = how much does the source here affect itself?
- $k(+1)$ = how much ... 1 step right affect here?

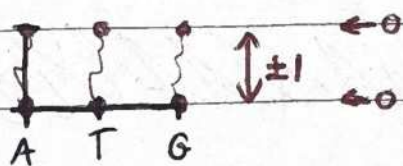
Now with this intuition time to compute the kernel values

"Math tends to reward you when you respect its symmetries" - 3B1B



∴ $dx = [-2, -1, 0, 1, 2]$

same applies vertically for dy



∴ $dy = [-1, 0, 1]$

I'm gonna do this systematically starting with $dy = 0$, then $dy = 1$, then $dy = -1$

① $D_y = 0 \rightarrow e^{(-0.3(0))} \cdot \cos(1.5(0) + 0.5) = \cos(0.5) = 0.8776 \dots \approx 0.8776$

$dx = -2 \rightarrow e^{(-0.5(2))} \cdot \cos(2(-2) + 0) = 0.3679 \times (-0.6531)(0.8776) = -0.2107$

$dx = -1 \rightarrow e^{(-0.5(1))} \cdot \cos(2(-1) + 0) = 0.6065 \times (-0.4161)(0.8776) = -0.2214$

$dx = 0 \rightarrow e^{(-0.5(0))} \cdot \cos(2(0) + 0) = 1 \times 1(0.8776) = 0.8776$

$dx = 1 \rightarrow e^{(-0.5(1))} \cdot \cos(2(1) + 0) = 0.6065 \times (-0.4161)(0.8776) = -0.2214$

$dx = 2 \rightarrow e^{(-0.5(2))} \cdot \cos(2(2) + 0) = 0.3679 \times (-0.6531)(0.8776) = -0.2107$

$k(dx, dy) = k(dx)k(dy)$

27/2/26

⑧ $D_y = 1 \rightarrow e^{(-0.3(1))} \cdot \cos(1.5 + .5) = e^{-.3}(\cos(2)) = \underline{-0.3083}$

$$D_x = -2 \rightarrow (0.3679)(-0.6531)(-0.3083) = 0.0741$$

$$\begin{aligned} D \times = -2 &\rightarrow (0.3679)(-0.6531)(-0.3083) \\ D \times = -1 &\rightarrow (0.6065)(-0.4161)(-0.3083) = 0.0778 \end{aligned}$$

$$(DX = 0) \rightarrow (1)(1)(-0,3083) = -0,3083$$

$\boxed{D8 = 0} \rightarrow (1)(1)(-0.3083) = -0.3083$
 $\boxed{D8 = 1} \rightarrow (0.6065)(-0.4161)(-0.3083) = 0.0778$

$$DX = 2 \rightarrow (0.3674)(-0.6531)(-0.3083) = 0.0741$$

① $D_y = -1 \rightarrow e^{(-0.3)} \cdot \cos(-1.5 + .5) = e^{-.3} \cos(-1) = \underline{0.4003}$

$$D_x = -2 \rightarrow (0.3679)(-0.6531)(0.4003) = -0.0962$$

$$(D_X = -1) \rightarrow (0.6065) \quad (-0.4161) \quad (0.4003) = -0.1010$$

$$(D_x = 0) \rightarrow (1)(1)(0.4003) = 0.4003$$

$$(D_x = 1) \rightarrow (0.6065) (-0.4161) (0.4003) = -0.100$$

$$Q_x = 2 \rightarrow (0.3679) (-0.6531) (0.4003) = -0.0962$$

Step 3 $\rightarrow$ Convolution

Now I need to find the new field values

$\frac{d}{dx} x^2 = 2x$
 $\frac{d}{dx} x^3 = 3x^2$
 $\frac{d}{dx} x^4 = 4x^3$
 $\frac{d}{dx} x^5 = 5x^4$
 $\frac{d}{dx} x^6 = 6x^5$
 $\frac{d}{dx} x^7 = 7x^6$
 $\frac{d}{dx} x^8 = 8x^7$
 $\frac{d}{dx} x^9 = 9x^8$
 $\frac{d}{dx} x^{10} = 10x^9$
 $\frac{d}{dx} x^{11} = 11x^{10}$
 $\frac{d}{dx} x^{12} = 12x^{11}$
 $\frac{d}{dx} x^{13} = 13x^{12}$
 $\frac{d}{dx} x^{14} = 14x^{13}$
 $\frac{d}{dx} x^{15} = 15x^{14}$
 $\frac{d}{dx} x^{16} = 16x^{15}$
 $\frac{d}{dx} x^{17} = 17x^{16}$
 $\frac{d}{dx} x^{18} = 18x^{17}$
 $\frac{d}{dx} x^{19} = 19x^{18}$
 $\frac{d}{dx} x^{20} = 20x^{19}$
 $\frac{d}{dx} x^{21} = 21x^{20}$
 $\frac{d}{dx} x^{22} = 22x^{21}$
 $\frac{d}{dx} x^{23} = 23x^{22}$
 $\frac{d}{dx} x^{24} = 24x^{23}$
 $\frac{d}{dx} x^{25} = 25x^{24}$
 $\frac{d}{dx} x^{26} = 26x^{25}$
 $\frac{d}{dx} x^{27} = 27x^{26}$
 $\frac{d}{dx} x^{28} = 28x^{27}$
 $\frac{d}{dx} x^{29} = 29x^{28}$
 $\frac{d}{dx} x^{30} = 30x^{29}$
 $\frac{d}{dx} x^{31} = 31x^{30}$
 $\frac{d}{dx} x^{32} = 32x^{31}$
 $\frac{d}{dx} x^{33} = 33x^{32}$
 $\frac{d}{dx} x^{34} = 34x^{33}$
 $\frac{d}{dx} x^{35} = 35x^{34}$
 $\frac{d}{dx} x^{36} = 36x^{35}$
 $\frac{d}{dx} x^{37} = 37x^{36}$
 $\frac{d}{dx} x^{38} = 38x^{37}$
 $\frac{d}{dx} x^{39} = 39x^{38}$
 $\frac{d}{dx} x^{40} = 40x^{39}$
 $\frac{d}{dx} x^{41} = 41x^{40}$
 $\frac{d}{dx} x^{42} = 42x^{41}$
 $\frac{d}{dx} x^{43} = 43x^{42}$
 $\frac{d}{dx} x^{44} = 44x^{43}$
 $\frac{d}{dx} x^{45} = 45x^{44}$
 $\frac{d}{dx} x^{46} = 46x^{45}$
 $\frac{d}{dx} x^{47} = 47x^{46}$
 $\frac{d}{dx} x^{48} = 48x^{47}$
 $\frac{d}{dx} x^{49} = 49x^{48}$
 $\frac{d}{dx} x^{50} = 50x^{49}$
 $\frac{d}{dx} x^{51} = 51x^{50}$
 $\frac{d}{dx} x^{52} = 52x^{51}$
 $\frac{d}{dx} x^{53} = 53x^{52}$
 $\frac{d}{dx} x^{54} = 54x^{53}$
 $\frac{d}{dx} x^{55} = 55x^{54}$
 $\frac{d}{dx} x^{56} = 56x^{55}$
 $\frac{d}{dx} x^{57} = 57x^{56}$
 $\frac{d}{dx} x^{58} = 58x^{57}$
 $\frac{d}{dx} x^{59} = 59x^{58}$
 $\frac{d}{dx} x^{60} = 60x^{59}$
 $\frac{d}{dx} x^{61} = 61x^{60}$
 $\frac{d}{dx} x^{62} = 62x^{61}$
 $\frac{d}{dx} x^{63} = 63x^{62}$
 $\frac{d}{dx} x^{64} = 64x^{63}$
 $\frac{d}{dx} x^{65} = 65x^{64}$
 $\frac{d}{dx} x^{66} = 66x^{65}$
 $\frac{d}{dx} x^{67} = 67x^{66}$
 $\frac{d}{dx} x^{68} = 68x^{67}$
 $\frac{d}{dx} x^{69} = 69x^{68}$
 $\frac{d}{dx} x^{70} = 70x^{69}$
 $\frac{d}{dx} x^{71} = 71x^{70}$
 $\frac{d}{dx} x^{72} = 72x^{71}$
 $\frac{d}{dx} x^{73} = 73x^{72}$
 $\frac{d}{dx} x^{74} = 74x^{73}$
 $\frac{d}{dx} x^{75} = 75x^{74}$
 $\frac{d}{dx} x^{76} = 76x^{75}$
 $\frac{d}{dx} x^{77} = 77x^{76}$
 $\frac{d}{dx} x^{78} = 78x^{77}$
 $\frac{d}{dx} x^{79} = 79x^{78}$
 $\frac{d}{dx} x^{80} = 80x^{79}$
 $\frac{d}{dx} x^{81} = 81x^{80}$
 $\frac{d}{dx} x^{82} = 82x^{81}$
 $\frac{d}{dx} x^{83} = 83x^{82}$
 $\frac{d}{dx} x^{84} = 84x^{83}$
 $\frac{d}{dx} x^{85} = 85x^{84}$
 $\frac{d}{dx} x^{86} = 86x^{85}$
 $\frac{d}{dx} x^{87} = 87x^{86}$
 $\frac{d}{dx} x^{88} = 88x^{87}$
 $\frac{d}{dx} x^{89} = 89x^{88}$
 $\frac{d}{dx} x^{90} = 90x^{89}$
 $\frac{d}{dx} x^{91} = 91x^{90}$
 $\frac{d}{dx} x^{92} = 92x^{91}$
 $\frac{d}{dx} x^{93} = 93x^{92}$
 $\frac{d}{dx} x^{94} = 94x^{93}$
 $\frac{d}{dx} x^{95} = 95x^{94}$
 $\frac{d}{dx} x^{96} = 96x^{95}$
 $\frac{d}{dx} x^{97} = 97x^{96}$
 $\frac{d}{dx} x^{98} = 98x^{97}$
 $\frac{d}{dx} x^{99} = 99x^{98}$
 $\frac{d}{dx} x^{100} = 100x^{99}$

I am not going to think of 2D convolutions in terms of their formula as that isn't ~~the~~ a smart way to approach this. I learned that convolutions can be represented as a multiplicative grid where you add like terms.

Example

$x^2 - 4x - 2$
 $2x^2$
 $-x$
 -1

$2x^4$	$-8x^3$	$-4x^2$
x^3	$4x^2$	$2x$
$-x^2$	$4x$	2

$$2x^4 - 9x^3 - x^2 + 6x + 2$$

2/9/26

083

I just need to extend this idea into the 3D space and build an analogy for it so I can do the calculations on pen and paper.

Well actually it would be 4d space? Wait that's perf! I can use the separability of my kernel ~~the~~ $k(dx, dy) = k(dx)k(dy)$ and treat it as 2 convolutions of 2D grids. This is amazing! That means I actually never needed to calculate $k(dx, dy)$ in the first place but $k(dy)$ and $k(dx)$. Damn this might save on computation too.

3

$$k(dx) \rightarrow k_x = \begin{matrix} & -2 & -1 & 0 & 1 & 2 \\ [-0.2403, & -0.2524, & 1, & -0.2524, & -0.2403] \end{matrix}$$

$$k(dy) \rightarrow k_y = \begin{matrix} & -1 & 0 & 1 \\ [0.4003, & 0.8776, & -0.3083] \end{matrix}$$

I mean notice how I was calculating same values throughout $Dy=n$ for Dx . $Dx=-2$ was the same for each operation so why the hell was I doing all that? $(0.3674)(-0.6531)$ appears twice per "dy" for all dy. It is a symmetric kernel too.

With this new insight I shall keep going...

I'm actually going to change the notation k_x and k_y into functions H and V representing Horizontal and Vertical influence. Easier this way I think :)

		pos0	pos1	pos2
$H(-2) \rightarrow$	-0.2403	-0.1922	0.1767	-0.2232
$H(-1) \rightarrow$	-0.2524	-0.2019	0.1767	-0.2232
$H(0) \rightarrow$	1	0.8	-0.7	0.9
$H(1) \rightarrow$	-0.2524	-0.2019	0.1767	-0.2232
$H(2) \rightarrow$	-0.2403	-0.1922	0.1767	-0.2232

Wrong! I realised midway through that dx and dy are directional and not positional so my table needs to be differently layed out
correction on next pg
↳

1/3/26

~~| | pos 0 | pos 1 | pos 2 |
|--------------------------------|-------|-------|-------|
| $H(0) \rightarrow 1$ | 0.8 | -0.7 | 0.9 |
| $H(\pm 1) \rightarrow -0.2524$ | 0.8 | -0.7 | 0.9 |
| $H(\pm 2) \rightarrow -0.2403$ | 0.8 | -0.7 | 0.9 |~~

OMG I'm being silly direction factor doesn't matter.
2D convolutions confused me. I'll just do as original.

still exists in diff positions silly

	0.8	-0.7	0.9
$H(-2) = -0.2403$	0.1922	0.1682	$-0.2163 \rightarrow 0.7604$ pos 0
$H(-1) = -0.2524$	-0.2019	0.1767	$-0.2272 \rightarrow -1.1291$ pos 1
$H(0) = 1$	0.8	-0.7	$0.9 \rightarrow 0.8845$ pos 2
$H(1) = -0.2524$	-0.2019	0.1767	-0.2272
$H(2) = -0.2403$	0.1922	0.1682	-0.2163

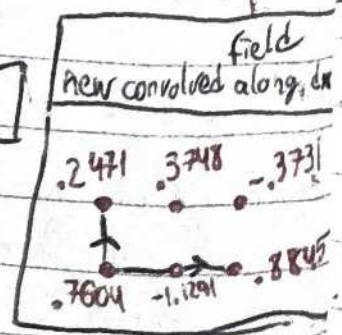
For Layer 0 $\rightarrow [0.7604, -1.1291, 0.8845]$

Accidentally had the wrong $h()$ value when punching into calc

	0.3	0.4	-0.2
-0.2403	-0.0757	-0.1010	$0.0505 \rightarrow 0.3748$ pos 0
-0.2524	-0.0721	-0.0961	$0.0481 \rightarrow 0.2471$ pos 1
1	0.3	0.4	-0.2 $\rightarrow -0.3731$ pos 2
-0.2524	-0.0721	-0.0961	0.0481
-0.2403	-0.0757	-0.1010	0.0505

For layer 1 $\rightarrow [0.2471, 0.3748, -0.3731]$

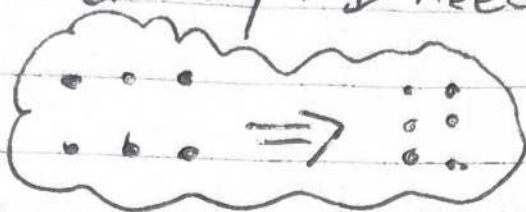
Now time to convolute vertically


~~| | 0.7604 | -1.1291 | 0.8845 |
|------------------|--------|---------|---------|
| $V(1) = 0.4003$ | 0.3044 | -0.4520 | 0.3341 |
| $V(0) = 0.8776$ | 0.6673 | -0.9409 | 0.7762 |
| $V(-1) = 0.3083$ | 0.2324 | 0.3481 | -0.2727 |~~

2/3/26

$$\begin{aligned}
 V(-1) &= 0.4003, 0.0489, 0.1500, -0.1494 \\
 V(0) &= 0.8776, 0.2169, 0.3289, -0.3274 \\
 V(1) &= -0.3083, -0.0762, -0.1156, 0.1150
 \end{aligned}$$

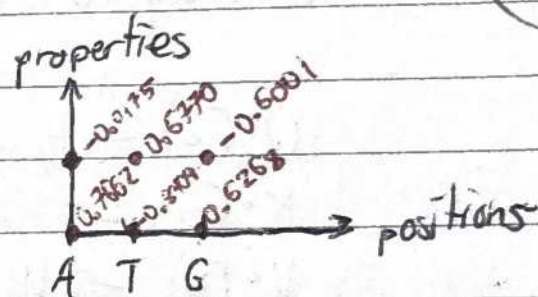
Convolute vertically! I need to respect that with grid layout



I was doing some weird hybrid of vertical and horizontal. ^{this} makes actual sense.

		0.7604, .2471	-1.1291, .3748	.8845, -3731
0.4003	V(-1)	0.3044 + 0.0989	-0.4520, 0.1500	0.3541, -0.1494
0.8776	V(0)	0.6673, 0.2169	-0.9109, 0.3289	0.7762, -0.3274
-0.3083	V(1)	-0.2344, -0.0762	0.3481, -0.1156	-0.2727, 0.1150

Layer 0	Layer 1
pos 0 → 0.7662	-0.0175
pos 1 → -0.8409	0.6770
pos 2 → 0.6268	-0.6001



This enough allowed me to visualise the process. Awesome!

I will explore the architecture on the next page.



18/3/26

Step by Step architecture

① QKV Projection Content $\rightarrow$ Initial Field

For each position "i", start with an embedding vector

Instead of projecting to a single vector, project to 3 distributions over properties:

$$\begin{aligned} Q_i &= \sigma(x_i W_Q) \in \mathbb{R}^P & \text{Where } W_Q, W_K \text{ map to probability} \\ K_i &= \sigma(x_i W_K) \in \mathbb{R}^P & \text{distributions over the } P \text{ properties using} \\ V_i &= x_i W_V \in \mathbb{R}^{P \times d_v} & \text{softmax.} \end{aligned}$$

W

W_V reshaped to P vectors of size d_v

A way to interpret this is:

$Q_i(p) \rightarrow$ How much should position "i" query property channel p
 $K_i(p) \rightarrow$ How ~~much strength~~ strongly does position "i" emit into property channel p
 $V_i(p) \rightarrow$ What value info does position "i" emit into property channel p

② Bilinear Scatter (Depositing $Q \otimes K$ onto Field)

Each ~~own~~ property channel p is actually a continuous dimension.
 So Q_i and K_i define probability densities over property space.

The initial field ~~a~~ becomes...

$O(T \cdot P)$ - still linear!

$$F_{\text{initial}}(x, p) = \sum_{i=1}^T [Q_i(p) \cdot K_i(p)] \cdot k_x(x-i)$$

Where k_x spreads along the sequence axis only (property spreading handled by convolution in a bit)

13/26 3 Wave Convolution

k_x was convoluted during scatter so to perform a 2D wave propagation using my kernel (separable property really coming in clutch) I just need

$$F_{\text{propagated}}(x, p) = \int F_{\text{initial}}(x, p') \cdot k_y(p - p') dp'$$

The k_x was already taken care of when spread each tokens "influence" to neighbouring x positions during scatter

4 Heads share info

Diff heads have Diff QKV projections, creating different initial fields...

$$L_{\text{mixed}}(x, p) = \sum_i [\underline{Q_i^{(h)}(p)} \cdot \underline{K_i^{(h)}(p)}] \cdot k_x(x - i)$$

Static coupling mixes these fields before propagation. (Interaction of fixed parameters in waves)...

$$L_{\text{mixed}}(x, p) = \sum_{h=1}^H \alpha_h L_{\text{initial}}^{(h)}(x, p)$$

Then just apply same k_y convolution to the mixed field.

5 Content Dependant Gating

V now comes into play. The gating mechanism uses the value vectors to modulate the field after propagation.

For each position i , scale entire propagated field at that x coordinate by the token's gating vector

$$L_{\text{final}}(i, p) = G_i(p) \cdot F_{\text{propagated}}(i, p) \quad \left| \begin{array}{l} \text{and for positions between tokens} \\ (x \text{ not integer), interpolate.} \end{array} \right.$$

1 8/3/26

⑥ Bilinear Gather

Finally, each token reads from the field using value vectors.

$$A_i = \sum_{p=1}^P V_i(p) \cdot F_{\text{final}}(i, p)$$

Where $V_i(p) \in \mathbb{R}^{d_v}$ and $F_{\text{final}}(i, p)$ is a scalar, so the product is a d_v -dimensional vector

Perf!

Computational Complexity

- 1 QKV projection $\rightarrow O(T \times d \times P) = \underline{O(T)}$ ✓
- 2 Bilinear scatter $\rightarrow O(T \times P \times \text{kernel width}) = \underline{O(T)}$ ✓
- 3 Multi-head coupling $\rightarrow O(H \times T \times P) = \underline{O(T)}$ ✓
- 4 Property convolution via FFT $\rightarrow O(T \times P \log P) = \underline{O(T)}$ ✓
- 5 Content gating $\rightarrow O(T \times P \times d) = \underline{O(T)}$ ✓
- 6 Apply gate $\rightarrow O(T \times P) = \underline{O(T)}$ ✓
- 7 Bilinear Gather $\rightarrow O(T \times P \times d_v) = \underline{O(T)}$ ✓

Total: $O(T \log T)$ from FFT step, which is the only superlinear (more than linear rate of growth in compute) part!

2D convolution is $O(T^2 P^2)$ but P is fixed, T varies and separability of my kernel makes it $O(T \log T)$. Really lucky that the formula from the paper generalised so nicely as a multivariable function. I think I'm ready to work on a prototype now.

So the New Paper dropped

I found an update Github repository for version 4.3.5 of Wave field LLM and it's everything I hoped it would become. They added so many cool additions such as HiPPO Harmonics, initializing each head with a diff frequency from step 0. Early layers get low frequencies (broad, long memory), later layers get high frequencies (sharp, local).

$$\omega_n = \pi(2n+1)/2$$

They also created a few visualisation tools such as the wave field building up token by token on a 3D graph and much more. Definetly will be fucking this!

Most importantly are the benchmarks of new model...

Model	Params	PPL	Accuracy	Complexity	Tokens/sec
Standard Transformer	17.5M	162.5	18.8%	$O(n^2)$	59,000
Spectre-Wave V4.3.5	22M	6.8	64.3%	$O(\log n)$	24,000

① Wiki-text-2, BPE 8k vocab, 20million tokens (8 epochs), seq_len=512, seed=42, RTX 3080. Both models trained identically on same data, optimized, learning rate schedule.

Sick! Convergence is also ~~class~~ class:

'Wave Field reaches Standard's final PPL in 1/4 the tokens

Milestone	Wave Field	Standard	Wave is faster by
PPL < 335	3M tokens	5M tokens	1.7x
PPL < 162	4M tokens	never	--
PPL < 100	5M tokens	never	--
PPL < 10	15M tokens	never	--
Final (20M)	6.8	162.5	23.4x better

1 11/3/26

To Do (Strategic Copying)

DNA-WAVE-FIELD

- README.md
- Requirements.txt
- src/

• venv
obviously included

- __init__.py

- core/

- __init__.py
 - wave_kernel.py → Copy + Modify
 - fft_optimizer.py → copy (its perf)
 - spectral_gate.py → Copy spectral gate
 - field_attention.py → My own 2D version

- dna/

- __init__.py
 - properties.py → fixed properties
 - synthetic.py → sequence generator
 - probing.py → probing infrastructure

- models/

- __init__.py
 - dna_wave_field.py → Main Model

- losses/

- __init__.py
 - tropical.py → tropical polynomials from insight

- tests/

- test_causality.py
 - test_probing.py
 - test_synthetic.py

- notebooks/

- probing_visualisation.ipynb

(I didn't like how the chain rule handled waves in gradient descent so converting to polynomials for accuracy)

3/26

Their code (1D) assumes field: (B, H, G) (Batch, heads, ^{Field pos})
I need field: (B, H, G, P) Batch, Heads, Field position, properties
(2D) $\nearrow$

I'll need to add a train.py and data/ folder to train of Data. First Data I'll be training on is E. Coli $\rightarrow$ very popular and well studied.

./data/ecoli.fa

3/26

So I ran the model for 1.5 hrs 7 epochs
And I got some weird results. The model results show something strange: all activations are near zero (0.0000). This means my model has learned to predict DNA sequences perfectly (loss near zero) but hasn't developed specialized feature/property detectors. It's using a different strategy!

from this I have learned that...

1. Perfect prediction $\neq$ interpretable representations.
2. The model learned to memorize local patterns rather than building abstract property representations
3. I need to encourage specialization through architecture (more layers) or training objectives